GEiPRS Vignette

Le Huang, Wujuan Zhong, Song Zhai, Judong Shen

2026-02-11

Introduction

GEiPRS is a package that is used to fit the group lasso or sparse group lasso on big genomics data. We assume the data are stored in .pgen/.pvar/.psam format by the PLINK library. In addition, it also requires the .pvar.zst file which can be generated using zstd. For example, `zstd yourdata.pvar` will generate `yourdata.pvar.zst`. The potential training/validation split can be specified with a separate column in the phenotype file.

Please note that the `geiprs` function in the GEiPRS package currently supports only the 'gaussian' family; other families are not supported at this time.

The most essential parameters in the core function `geiprs` include:

- `genotype.pfile`: the PLINK 2.0 pgen file that contains genotype. We assume the existence of genotype.pfile.{pgen,pvar.zst,psam}.
- `phenotype.file`: the path of the file that contains the phenotype values and can be read as a table.
- `phenotype`: the name of the phenotype. Must be the same as the corresponding column name in the phenotype file.
- `env`: the name of the environment variable. Must be the same as the corresponding column name in the phenotype file. `env` is required to run the group lasso or sparse group lasso.
- `tau`: the sparse group lasso mixing parameters ($0 \leq tau < 1$). If tau = 0, it is group lasso penalty. The lasso penalty can be approximated by using tau close to 1 (e.g., 0.999). `tau` is required to run the group lasso or sparse group lasso.
- `covariates`: a character vector containing the names of the covariates included in the model (group lasso or spare group lasso) fitting, whose coefficients will not be penalized. The names must exist in the column names of the phenotype file.
- `family`: the type of the phenotype: "gaussian". Currently only "gaussian" family is supported for applying the group lasso or sparse group lasso in the geiprs function. Default is "gaussian".
- `split.col`: the column name in the phenotype file that specifies the membership of individuals to the training or the validation set. The individuals marked as `"train"` and `"val"` will be treated as the training and validation set, respectively. When specified, the model performance is evaluated on both the training and the validation sets.
- `mem`: Memory (MB) available for the program. It tells PLINK 2.0 the amount of memory it can harness for the computation. IMPORTANT if using a job scheduler.

Some additional important parameters for model building include:

- `nlambda`: the number of lambda values on the solution path.
- `lambda.min.ratio`: the ratio of the minimum lambda considered versus the maximum lambda that makes all penalized coefficients zero.
- `p.factor`: a named vector of separate penalty factors applied to each coefficient. This is a number that multiplies lambda to allow different shrinkage. If not provided, default is 1 for all variables. Otherwise should be complete and positive for all variables.

Other parameters can be specified in a config list object, such as `missing.rate`, `MAF.thresh`, `nCores`, `num.groups.batch` (batch size M of the GITLABS algorithm for group lasso and sparse group lasso), `save`

(whether to save intermediate results), `results.dir`, `prevIter` (when starting from the middle). More details can be seen in the function documentation. In particular, If we want to recover results and continue the procedure from a previous job, we should have `save = TRUE` and specify `prevIter` with the index of the last successful (and saved) iteration.

GEiPRS depends on two other programs **plink2** and **zstdcat**. If they are not already on the system serach path, it is important to specify their locations in the `configs` object and pass it to `geiprs`.

```
configs <- list(
  # results.dir = "PATH/TO/SAVE/DIR",  # needed when saving intermediate results
  # save = TRUE,  # save intermediate results per iteration (default FALSE)
  # nCores = 16,  # number of cores available (default 1)
  # niter = 100,  # max number of iterations (default 50)
  # prevIter = 15,  # if we want to start from some iteration saved in results.dir
  # early.stopping = FALSE,  # whether to stop based on validation performance (default TRUE)
  # verbose = TRUE,  # print intermediate messages (default FALSE)
  plink2.path = "plink2",   # path to plink2 program
  zstdcat.path = "zstdcat"  # path to zstdcat program
)
# check if the provided paths are valid
for (name in names(configs)) {
  tryCatch(system(paste(configs[[name]], "-h"), ignore.stdout = T),
    condition = function(e) cat("Please add", configs[[name]], "to PATH, or modify the path in the confi
  )
}
```

Example

We utilize the example data to illustrate the application of the GEiPRS package in constructing the genotype main effect-based polygenic risk score (PRS_G) and the genotype-environment interaction effect-based polygenic risk score (PRS_{GEI}) using the sparse group lasso method.

```
library(geiprs)
```

For the following script, when `split.col` is specified, validation metric will automatically be computed. Moreover, by default, early stopping is adopted to stop the process based on the validation metric (the extent controlled by `stopping.lag` in `configs` object).

```
genotype.pfile <- file.path(system.file("extdata", package = "geiprs"), "example.trainingANDvalidation")
phenotype.file <- system.file("extdata", "example.phe", package = "geiprs")
phenotype <- "resid"
env <- "envir"

fit_geiprs <- geiprs(
  genotype.pfile = genotype.pfile,
  phenotype.file = phenotype.file,
  phenotype = phenotype,
  env = env,
  tau = 0.5,
  split.col = "split",  # split column name in phenotype.file with train/val/test labels
  # mem = 128000,  # amount of memory available (MB), recommended
  configs = configs,
  family = "gaussian"
)  # we hide the intermediate messages
```

The intercept and coefficients can be extracted by `fit_geiprs$a0` and `fit_geiprs$beta`. It also saves the evaluation metric, which by default is R^2 for the Gaussian family.

```
fit_geiprs$metric.train %>% na.omit()
#>  [1] 0.008461921 0.008862384 0.009194857 0.009470881 0.009700041 0.009890293
#>  [7] 0.010048245 0.011032154 0.012302843 0.014015410 0.015008158 0.016645767
#> [13] 0.018190636 0.020204719 0.021965781 0.023775334 0.025919799 0.028847561
#> [19] 0.031890599 0.035460488 0.041692037 0.047256194 0.053562951 0.061154862
#> [25] 0.070468005 0.081997302 0.094645266 0.108257568 0.123175322
#> attr(,"na.action")
#>  [1]  30  31  32  33  34  35  36  37  38  39  40  41  42  43  44  45  46  47  48
#> [20]  49  50  51  52  53  54  55  56  57  58  59  60  61  62  63  64  65  66  67
#> [39]  68  69  70  71  72  73  74  75  76  77  78  79  80  81  82  83  84  85  86
#> [58]  87  88  89  90  91  92  93  94  95  96  97  98  99 100
#> attr(,"class")
#> [1] "omit"
fit_geiprs$metric.val %>% na.omit()
#>  [1] 0.006373289 0.006608089 0.006789217 0.006927011 0.007029947 0.007104961
#>  [7] 0.007157722 0.007783825 0.008608358 0.009705663 0.010013090 0.010520377
#> [13] 0.011428409 0.012675864 0.013706372 0.014588136 0.015603121 0.017032251
#> [19] 0.018411689 0.020174339 0.023717423 0.025382932 0.026471078 0.027475441
#> [25] 0.028078560 0.028914028 0.029815378 0.028715901 0.027943687
#> attr(,"na.action")
#>  [1]  30  31  32  33  34  35  36  37  38  39  40  41  42  43  44  45  46  47  48
#> [20]  49  50  51  52  53  54  55  56  57  58  59  60  61  62  63  64  65  66  67
#> [39]  68  69  70  71  72  73  74  75  76  77  78  79  80  81  82  83  84  85  86
#> [58]  87  88  89  90  91  92  93  94  95  96  97  98  99 100
#> attr(,"class")
#> [1] "omit"
```

Next, we can get the optimal `idx` based on the validation metric. For the testing set, we can use the `predict_geiprs` function to predict the PRS_G and PRS_{GEI} with the fitted object `fit_geiprs`, based on the optimal `idx`.

```
optimal_idx <- which.max(fit_geiprs$metric.val)

test.genotype.pfile <- file.path(system.file("extdata", package = "geiprs"), "example.testing")


pred_geiprs <- predict_geiprs(
  fit = fit_geiprs,
  new_genotype_file = test.genotype.pfile ,
  new_phenotype_file = phenotype.file,
  phenotype = phenotype,
  env = env,
  split_col = "split",
  split_name = c("test"),
  idx = optimal_idx,
  configs = configs)
#> Warning in checkMissingPhenoWarning(phe.master, phe.no.missing.IDs): We detected missing values for

names(pred_geiprs)
#> [1] "prediction"         "prs_g"              "prs_ge"
#> [4] "response"           "env_val"            "metric"
#> [7] "metric.prs"         "metric.prs.logpval"
```

Now we can get predicted PRS_G and PRS_{GEI}.

```
cat("Predicted PRS_G:", "\n")
#> Predicted PRS_G:
pred_prs_g <- pred_geiprs[['prs_g']][['test']]
print(head(pred_prs_g))
#>                              s26
#> 0-unrelatedBritish11 0.076431798
#> 0-unrelatedBritish13 0.065903629
#> 0-unrelatedBritish16 0.064349067
#> 0-unrelatedBritish20 0.005729977
#> 0-unrelatedBritish28 0.042642733
#> 0-unrelatedBritish29 0.007233464

cat("Predicted PRS_GEI:", "\n")
#> Predicted PRS_GEI:
pred_prs_gei <- pred_geiprs[['prs_ge']][['test']]
print(head(pred_prs_gei))
#>                               s26
#> 0-unrelatedBritish11  0.4401397
#> 0-unrelatedBritish13 -0.1572999
#> 0-unrelatedBritish16  0.1790730
#> 0-unrelatedBritish20  0.1202063
#> 0-unrelatedBritish28  0.2163110
#> 0-unrelatedBritish29 -0.1257624
```

R^2 explained by envir, PRS_G, and envir * PRS_{GEI} is stored in `pred_geiprs[['metric.prs']][['test']]`.

```
pred_geiprs[['metric.prs']][['test']]
#> [1] 0.04388405
```

For the regression model: phenotype ~ envir + PRS_G + envir * PRS_{GEI}, the -log10(P value of the envir * PRS_{GEI}) is stored in `pred_geiprs[['metric.prs.logpval']][['test']]`.

```
pred_geiprs[['metric.prs.logpval']][['test']]
#> [1] 66.84141
```